

Official Flutter Plugin Ecosystem: Coverage Analysis and Minimum Project Delta

A source-validated reassessment of what the official Flutter AI/plugin ecosystem already ships, exactly which of the project's architectural decisions it already enforces, and what the smallest possible custom Flutter governance layer must look like on top.

Executive Summary

 **The official ecosystem covers most of the architecture — but not the state-management choice**

The Flutter team's flutter/agent-plugins repository already ships an `flutter-apply-architecture-best-practices` skill that mandates **MVVM + layered UI/Logic/Data + Repository pattern + freezed models + provider for DI**. However, the same skill explicitly uses `ChangeNotifier` + `ListenableBuilder`, and the official rules.md says *"Prefer Flutter's built-in state management solutions. Do not use a third-party package unless explicitly requested"*. The Bloc/Cubit decision is therefore **a project-specific delta**, not a Flutter-endorsed default.

 **Claude Code plugins CANNOT ship rules — Devin plugins CAN**

Claude Code's plugin spec is explicit: *"A `CLAUDE.md` file at the plugin root is not loaded as project context. Plugins contribute context through skills, agents, and hooks rather than `CLAUDE.md`"*. Devin plugins, in contrast, ship an `AGENTS.md` at the plugin root that *"is injected as an always-on rule in every session"*. This asymmetry is the single most important design constraint for the custom governance layer.

Multi-plugin coexistence is safe by design — everything is namespaced

Plugin skills become `/plugin-name:skill-name`, agents become `plugin-name:agent-name`, MCP tools become `mcp__plugin__`. So the memory/learning plugin and the Flutter plugin cannot collide on any name.

A full custom Flutter plugin is NOT required

The optimal shape is to **install the official plugin as-is** and add a small project-level governance layer on top: (1) a `./.claude/CLAUDE.md` with the Bloc-vs-ChangeNotifier delta, (2) path-scoped rules under `./.claude/rules/`, (3) one small optional custom plugin only if you need cross-project reuse of Bloc-specific skills.

Two-tier design: Project Brain (universal) + Flutter (official) + tiny delta

The Project Brain plugin handles memory/learning across all languages. The Flutter plugin ships officially from `flutter/agent-plugins`. The delta lives in the project itself (`CLAUDE.md/AGENTS.md` + `rules/`) — not in a new plugin — because the delta is *project-specific*, not *Flutter-ecosystem-specific*.

[6](#) [7](#) [5](#) [1](#)

1. What the Official Flutter Ecosystem Already Provides

1.1 The `flutter/agent-plugins` repository

The Flutter team maintains a single repository, `flutter/agent-plugins` (2.7k stars, BSD-3-Clause), described as *"A collection of plugins designed to extend AI agent capabilities for Flutter development. These plugins bundle together skills, MCP server configuration, and rules to provide tailored workflows and instructions for happy path Flutter development."* [11](#)

The repo now ships **three plugin flavors from the same source** [11](#):

Claude Code plugin

[.claude-plugin/](#)

marketplace.json + plugin.json —
installed via `claude plugin install
dart-flutter@dart-flutter`

Codex plugin

[.codex-plugin/](#)

Installed via
`codex plugin add dart-
flutter@dart-flutter`

Cursor plugin

[.cursor-plugin/](#)

Ships alongside the others; the universal install path is the `npx skills` command

[11](#)

The repository also contains: [11](#)

- `.mcp.json` at the root — the pre-baked MCP server config so installing the plugin also wires the Dart & Flutter MCP server.
- `resources/` — example prompts.
- `skills/` — the actual skill catalog.
- `tool/dart_skills_lint/` — a Dart linter for skill quality (v0.4.0). [11](#)

1.2 Installation methods

Three officially documented paths: [11](#)

```
# Universal (writes to .agents/skills/)
npx skills@1.5.17 add flutter/agent-plugins --skill '*' --agent universal --yes

# Claude Code
claude plugin marketplace add flutter/agent-plugins
claude plugin install dart-flutter@dart-flutter

# Codex
codex plugin marketplace add flutter/agent-plugins
codex plugin add dart-flutter@dart-flutter
```

Updates are just: `npx skills@1.5.17 update` [11](#)

The **Dart skills repository** ([dart-lang/skills](https://github.com/dart-lang/skills)) is a separate but complementary install: `npx skills add dart-lang/skills --skill '*' --agent universal 10`.

1.3 The complete Flutter skill catalog (10 skills)

flutter-apply-architecture-best-practices

"Architects a Flutter application using the recommended layered approach (UI, Logic, Data). Use when structuring a new project or refactoring for scalability."

flutter-add-widget-test

"Implement a component-level test using `WidgetTester` to verify UI rendering and user interactions."

flutter-add-integration-test

"Configures Flutter Driver for app interaction and converts MCP actions into permanent integration tests."

flutter-add-widget-preview

"Adds interactive widget previews to the project using the `previews.dart` system."

flutter-build-responsive-layout

"Use `LayoutBuilder`, `MediaQuery`, or `Expanded/Flexible` to create a layout that adapts to different screen sizes."

flutter-fix-layout-issues

"Fixes Flutter layout errors (overflows, unbounded constraints) using Dart and Flutter MCP tools."

flutter-implement-json-serialization

"Create model classes with `fromJson` and `toJson` methods using `dart:convert`." Uses manual serialization, not `freezed`

flutter-setup-declarative-routing

"Configure `MaterialApp.router` using a package like `go_router` for advanced URL-based navigation."

flutter-setup-localization

"Add `flutter_localizations` and `intl` dependencies, enable `generate: true`, and create `l10n.yaml`."

flutter-use-http-package

"Use the `http` package to execute GET, POST, PUT, or DELETE requests."

[11](#) [13](#) [14](#)

1.4 Complementary Dart skill catalog (12 skills)

Installed via `dart-lang/skills`: [6](#)

- `dart-add-unit-test`, `dart-generate-test-mocks`, `dart-collect-coverage`
- `dart-run-static-analysis`, `dart-fix-runtime-errors`
- `dart-resolve-package-conflicts`, `dart-migrate-to-checks-package`
- `dart-use-pattern-matching`, `dart-use-primary-constructors`
- `dart-build-cli-app`, `dart-setup-ffi-assets`, `dart-use-ffigen`

1.5 Official AI Rules (docs.flutter.dev/ai/ai-rules)

The Flutter team publishes **four sizes of the same rules file**: [9](#)

File	Size	Use case
<code>rules.md</code>	Comprehensive master	Full context
<code>rules_10k.md</code>	<10k chars	Stricter limits
<code>rules_4k.md</code>	<4k chars	GitHub Copilot
<code>rules_1k.md</code>	<1k chars	Very strict limits

Per-tool file naming conventions are officially documented: [9](#)

- Claude Code: `CLAUDE.md` (no hard limit)
- Cursor: `AGENTS.md`
- Gemini CLI: `GEMINI.md`

- Antigravity: `.agent/rules/<rule-name>.md` (12k hard cap)
- GitHub Copilot: `.github/copilot-instructions.md` (~4k)
- JetBrains AI: `.junie/guidelines.md`
- VS Code: `.instructions.md`

The workflow the docs recommend is: download `rules.md`, feed to an LLM, ask it to reformat for your editor, verify [9](#).

But note: `rules.md` is a template you copy into your project — it is not automatically installed by any plugin. This is explicitly separate from the plugin path.

1.6 Dart & Flutter MCP Server (docs.flutter.dev/ai/mcp-server)

Currently experimental (Dart 3.9+). Started with `dart mcp-server` [8](#). Capabilities directly relevant to architecture enforcement: [8](#)

- Analyze and fix errors in your code
- Resolve symbols to elements, fetch documentation and signatures
- Introspect and interact with the running application
- Search pub.dev for the best package
- Manage `pubspec.yaml` dependencies
- Run tests and analyze results
- Format code with the same formatter as `dart format`

The MCP server is bundled with the Claude Code plugin via `.mcp.json` at the repo root [11](#), so installing `dart-flutter@dart-flutter` also configures the MCP server. Officially supported clients include Antigravity, Gemini CLI, Cursor, and GitHub Copilot [8](#).

2. Decision-by-Decision Coverage Matrix

For each architectural decision the project has already adopted, this section states whether the official ecosystem already guides, enforces, scaffolds, or automates it — and what additional governance (if any) is required.

2.1 Master coverage matrix

Adopted Decision	Official skill/ rule that covers it	What it says	Delta required?
MVVM architecture	<code>flutter-apply-architecture-best-practices</code>	<i>"Implement the MVVM (Model-View-ViewModel) pattern to manage UI state and logic"</i> 6	✗ None — fully covered
Layered UI/ Logic/Data	Same skill + official rules.md	<i>"Enforce strict Separation of Concerns by dividing the application into distinct layers"</i> 6 7	✗ None — fully covered
Repository pattern as SSOT	Same skill	<i>"Implement the Repository pattern to isolate data access logic and create a single source of truth"</i> 6	✗ None — fully covered
Optional Domain Layer / Use Cases	Same skill	<i>"Implement this layer only if the application contains complex business logic that clutters the ViewModel"</i> 6	✗ None — fully covered

Adopted Decision	Official skill/rule that covers it	What it says	Delta required?
Immutable models with Freezed	Same skill Step 1	"Create immutable data classes for the feature using <code>freezed</code> or <code>built_value</code> " 6	✗ None — fully covered
Compass-style folder structure	Same skill Project Structure	Shows exactly the same hybrid layout: <code>data/models</code> , <code>data/repositories</code> , <code>data/services</code> , <code>domain/models</code> , <code>domain/use_cases</code> , <code>ui/core</code> , <code>ui/features/<f></code> , <code>{view_models, views}</code> 6	✗ None — fully covered
Navigation with <code>go_router</code>	<code>flutter-setup-declarative-routing</code> + <code>rules.md</code>	"Use the <code>go_router</code> package for declarative routing" 14 7	✗ None — fully covered
Widget tests	<code>flutter-add-widget-test</code>	Uses <code>WidgetTester</code> for component-level tests 11	✗ None — fully covered

Adopted Decision	Official skill/ rule that covers it	What it says	Delta required?
Integration tests	<code>flutter-add-integration-test</code>	Uses <code>integration_test</code> package + Flutter Driver 11	✗ None — fully covered
Unit tests / mocks	<code>dart-add-unit-test</code> , <code>dart-generate-test-mocks</code>	From <code>dart-lang/skills</code> 6	✗ None — fully covered
Coverage	<code>dart-collect-coverage</code>	From <code>dart-lang/skills</code> 6	✗ None — fully covered
Static analysis	<code>dart-run-static-analysis</code> , <code>dart-fix-runtime-errors</code>	From <code>dart-lang/skills</code> 6	✗ None — fully covered
Naming conventions	rules.md	"PascalCase for classes, camelCase for members, snake_case for files" 7	✗ None — fully covered
Dependency Injection	<code>flutter-apply-architecture-best-practices</code> Step 7 + rules.md	"Register in the DI container (e.g., <code>provider</code> or <code>get_it</code>)" 7 6	⚠ Minor — pick one and document

Adopted Decision	Official skill/rule that covers it	What it says	Delta required?
Bloc/Cubit as state manager	NOT COVERED	Official skill uses <code>ChangeNotifier</code> + <code>ListenableBuilder</code> ⁶ ; rules.md says <i>"Prefer Flutter's built-in state management. Do not use a third-party package unless explicitly requested"</i> ⁷	✅ Delta required
Freezed for state/events	rules.md prefers <code>dart:convert</code> in <code>flutter- implement-json-serialization</code> ; architecture skill mentions freezed for domain models only	Architecture skill: <i>"Create immutable data classes for the feature using <code>freezed</code> or <code>built_value</code>"</i> ⁶ ; JSON skill uses manual <code>dart:convert</code> ¹³	⚠️ Delta — extend freezed use to Bloc states/events
very_good_analyses lints	Not covered	Not shipped in either skills repo	✅ Delta required
Three build flavors (dev/staging/prod)	Not covered	Not part of any official skill	✅ Delta required

2.2 Reading of the matrix

Out of the 17 adopted decisions:

- **13 are fully covered** by installing `flutter/agent-plugins` + `dart-lang/skills`.
- **1 requires a minor clarification** (which DI package).
- **3 require a real project-specific delta:** (a) Bloc/Cubit choice, (b) using freezed for Bloc states/events (not just domain models), (c) `very_good_analysis`, (d) build flavors.

Everything else is authored, versioned, and updated for you by the Flutter team.

3. The Bloc/Cubit Delta in Detail

This is the single most important gap between the official ecosystem and the project's decisions.

3.1 What the official architecture skill actually mandates

The `flutter-apply-architecture-best-practices` SKILL.md, in its "UI Layer" section: [6](#)

"Implement the MVVM (Model-View-ViewModel) pattern to manage UI state and logic.

- *Views: Write reusable, lean widgets. Restrict logic in Views to UI-specific operations (e.g., animations, layout constraints, simple routing). Pass all required data from the ViewModel.*
- ***ViewModels: Manage UI state and handle user interactions. Extend `ChangeNotifier` (or use `Listenable`) to expose state.** Expose immutable state snapshots to the View. Inject Repositories into ViewModels via the constructor."*

The Step-by-Step workflow is explicit: [6](#)

- Step 5: *"Implement the ViewModel. Create the ViewModel extending `ChangeNotifier`. Inject required Repositories/Use Cases. Expose immutable state and command methods."*
- Step 6: *"Implement the View. Create the UI widget. Use `ListenableBuilder` or `AnimatedBuilder` to listen to ViewModel changes."*
- Step 7: *"Inject Dependencies. Register the new Service, Repository, and ViewModel in the dependency injection container (e.g., `provider` or `get_it`)."*

The code example in the same file uses `class ProfileViewModel extends ChangeNotifier`, calls `notifyListeners()`, and renders via `ListenableBuilder` [6](#).

3.2 What the official rules.md mandates

From `flutter/flutter/docs/rules/rules.md`, the State Management section: [7](#)

"Built-in Solutions: Prefer Flutter's built-in state management solutions. Do not use a third-party package unless explicitly requested."

The rules then cascade through: `Streams / StreamBuilder`, `ValueNotifier` + `ValueListenableBuilder`, `ChangeNotifier`, `ListenableBuilder`, then **MVVM** as the pattern for anything larger. `provider` is only allowed for DI when "explicitly requested" — and even then the note says *"this document generally defaults against third-party packages for state management unless explicitly requested"* [7](#).

3.3 What this means for the project

Choosing Bloc/Cubit is a **legitimate but explicit override** of the Flutter defaults. The docs allow it (that's exactly what "unless explicitly requested" means), but the AI, following the official skill and rules verbatim, will produce `ChangeNotifier`-based ViewModels.

The delta must therefore:

1. **State the exception explicitly** at the project level so the AI knows to override the default.

2. **Reframe the MVVM pattern** so "ViewModel" == "Bloc/Cubit" in the project.
3. **Extend the flutter-apply-architecture-best-practices workflow** so Step 5 says "Implement the Cubit/Bloc" and Step 6 says "Use `BlocBuilder` to listen".
4. **Extend frozen use** to Bloc states and events (not just domain models).
5. **Publish an ADR** ("ADR-0001: State management is Bloc/Cubit, overriding the Flutter default of ChangeNotifier") so the decision is auditable.

None of this requires a new plugin. It's ~50 lines of markdown at the project root.

4. Plugin Composition: Documented Behavior

You asked what happens when the official Flutter plugin and the Project Brain plugin coexist, especially around rules, CLAUDE.md, AGENTS.md, and MCP. All answers below are from the official Claude Code and Devin references — no assumptions.

4.1 Claude Code composition rules

CLAUDE.md from plugins — the critical rule. The plugins reference states, in absolute terms: [5](#)

"A `CLAUDE.md` file at the plugin root is not loaded as project context. Plugins contribute context through skills, agents, and hooks rather than CLAUDE.md. To ship instructions that load into Claude's context, put them in a `skill`."

Consequence: two Claude Code plugins can each have a `CLAUDE.md` file at their root — **neither will affect the user's context**. There is no merge, no conflict, no overwrite — the files are inert. Guidance ships as skills.

Where CLAUDE.md content actually loads from. The memory documentation lists the four locations in load order, broadest to most specific: [4](#)

1. **Managed policy** — `/Library/Application Support/ClaudeCode/CLAUDE.md` (macOS), `/etc/claude-code/CLAUDE.md` (Linux/WSL), `C:\Program Files\ClaudeCode\CLAUDE.md` (Windows). Cannot be excluded by users.
2. **User instructions** — `~/.claude/CLAUDE.md`.

3. **Project instructions** — `./CLAUDE.md` or `./.claude/CLAUDE.md`.

4. **Local instructions** — `./CLAUDE.local.md` (personal, gitignored).

Plus `./.claude/rules/*.md` for path-scoped rules. All `.md` files under `./.claude/rules/` are discovered recursively, so `./.claude/rules/frontend/*.md` and `./.claude/rules/backend/*.md` co-exist naturally [4](#).

Path-scoped rules. Rules with a YAML frontmatter `paths:` field only load when Claude accesses matching files — this is the officially recommended way to keep context clean: [4](#)

```
---
paths:
  - "lib/ui/**/*.dart"
---
# UI Layer Rules
- Views may contain no logic beyond routing/layout/animation.
- ViewModels are Cubit/Bloc instances (project override – see ADR-0001).
```

Skills. Plugin skills are namespaced by plugin name — a skill named `hello` in plugin `my-plugin` becomes `/my-plugin:hello` [12](#). Two plugins each defining a `flutter-add-feature` skill would coexist as `/plugin-a:flutter-add-feature` and `/plugin-b:flutter-add-feature`.

Agents. Agents from plugins appear in the `@`-mention typeahead under their scoped name, e.g. `my-plugin:code-reviewer` [5](#).

Hooks, MCP servers, LSP servers. These *"own merge rules"* [5](#) (from the "Path behavior rules" section). Specifically for MCP: plugin MCP servers *"start automatically when the plugin is enabled"* and *"appear as standard MCP tools in Claude's toolkit"* [5](#). Tool names get scoped: `mcp__plugin__<plugin-name>__<server-name>__<tool>` [5](#).

Practical consequence. Two plugins can each ship a `.mcp.json` and both servers will start. They cannot collide because each server's tools are namespaced by plugin name. Hooks likewise fire independently.

4.2 Devin composition rules

Devin's model is **explicitly different** from Claude Code's on rules. From the Devin plugin overview: [1](#)

"A plugin can also ship rules: an `AGENTS.md` at the plugin root is injected as an always-on rule in every session, alongside your project's own rules. Markdown files in a `rules/` folder are loaded too, with the same `trigger` frontmatter and activation types as Windsurf rules."

So the same directory that Claude Code silently ignores, Devin loads as active rules. This means:

- If you ship an `AGENTS.md` in the Flutter plugin, **Devin will inject it** as always-on rules; Claude Code will ignore it.
- If both the Project Brain plugin and Flutter plugin ship `AGENTS.md`, Devin loads both.
- Load order and combination follow Devin's four-level authority chain: **Enterprise > Org > Repo > User** ¹.
- Within a level, `forbiddenPlugins` (deny) always wins over `optionalPlugins` (allow).
- Across levels, higher authority always wins.

Devin also natively reads `.claude/` content when configured: ²

```
"read_config_from": {  
  "agents_standard": true,  
  "cursor": true,  
  "windsurf": true,  
  "claude": true  
}
```

So `~/.claude/CLAUDE.md` is picked up as a Devin global rule too ².

4.3 Cross-tool coexistence summary

CLAUDE.md at plugin root

Claude: ignored

Not loaded as project context.

AGENTS.md at plugin root

Devin: always-on

Injected as always-on rule.

Skills from multiple plugins

Namespaced

All become `/plugin:skill` — no collisions.

MCP servers from multiple plugins

Namespaced tools

`mcp__plugin__` .

Hooks from multiple plugins

All fire

Own merge rules — plugins compose.

Project `./CLAUDE.md`
+ `./claude/rules/`

Primary layer

Where architecture delta must live for Claude Code.

[5](#) [1](#) [12](#) [4](#)

5. Reassessment: Do We Still Need a Custom Flutter Plugin?

Three options, ranked by effort vs. reuse.

5.1 Option A — Official plugin only, no project delta

- Install `flutter/agent-plugins` and `dart-lang/skills` .
- Accept `ChangeNotifier` as the state manager.
- Accept manual `dart:convert` for JSON.

Verdict: Rejected. This forfeits the Bloc/Cubit decision that the project deliberately made.

5.2 Option B — Official plugin + project-level governance layer (recommended)

- Install `flutter/agent-plugins` and `dart-lang/skills` (Claude Code plugin route).

- Add project-level:

- `./.claude/CLAUDE.md` — small (few hundred lines), citing overrides and ADRs.
- `./AGENTS.md` — mirror content for Devin (auto-loaded).
- `./.claude/rules/state-management.md` — path-scoped rule (`paths: ["lib/ui/**/*.*.dart", "test/ui/**/*.*.dart"]`) that says "use Bloc/Cubit, not ChangeNotifier".
- `./.claude/rules/models.md` — extends freezed use to Bloc states/events.
- `./analysis_options.yaml` — includes `package:very_good_analysis/analysis_options.yaml`.
- `./docs/adr/ADR-0001-state-management.md` — the auditable decision record.

Verdict:  **This is the recommended shape.** It reuses everything the Flutter team ships, updates automatically via `npm skills update` or `/plugin marketplace update`, and encodes the delta once at the project root.

5.3 Option C — Custom Flutter plugin bundling delta + reuse across projects

Same as Option B, but the delta lives inside a small custom plugin (call it `flutter-bloc-delta`) that:

- Ships an `AGENTS.md` (auto-loaded by Devin) ¹
- Ships one Skill: `flutter-apply-architecture-with-bloc` that **references** the official skill but overrides Step 5–6 ⁵.
- Declares `requiredPlugins` in a Devin manifest so installing your plugin also installs the Flutter one ¹.

Verdict: Only justified when:

- You will spawn ≥ 3 Flutter projects with the same Bloc override.
- You want cross-project version pinning (bump the plugin, all projects follow).
- You want to enforce the delta via `forbiddenPlugins` or org-level Devin manifests.

Otherwise the maintenance overhead outweighs the reuse.

5.4 Recommended architecture

The optimal composition is: Project Brain plugin + flutter/agent-plugins + project-level governance layer, with an optional custom "bloc-delta" plugin only when reuse across projects becomes real.

The Project Brain plugin handles memory/learning across all languages and stacks (its stated scope), the Flutter plugin ships officially from the Flutter team, and the delta lives in the project because the delta is project-specific — not Flutter-ecosystem-specific.

6. The Smallest Possible Flutter Delta

Based on §2 and §3, the delta is small and enumerable. Everything below is content authored by the project — everything not below comes from the official plugin.

6.1 Files to add to a new Flutter project

```
my_app/
├── .claude/
│   ├── CLAUDE.md                # Project overrides + ADR references
│   └── rules/
│       ├── state-management.md   # paths: lib/ui/**/*.dart → Bloc/Cubit
│       ├── models.md            # paths: lib/**/*.dart → freezed everywhere
│       ├── linting.md           # (no paths) → very_good_analysis
│       └── flavors.md           # (no paths) → dev/staging/prod entry points
├── AGENTS.md                   # Mirror of CLAUDE.md for Devin
├── analysis_options.yaml        # include: package:very_good_analysis/analysis_options.yaml
├── pubspec.yaml                # + flutter_bloc, bloc_test, freezed, freezed_annotation,
│                               #   json_annotation, json_serializable, build_runner,
│                               #   very_good_analysis, go_router
├── lib/
│   ├── main.dart
│   ├── main_development.dart
│   ├── main_staging.dart
│   └── main_production.dart
└── docs/adr/
    ├── ADR-0001-state-management.md # Override: Bloc/Cubit, not ChangeNotifier
    ├── ADR-0002-linting.md          # very_good_analysis
    └── ADR-0003-flavors.md          # 3 entry points
```

6.2 The .claude/CLAUDE.md content (skeleton)

Everything Claude needs to know at session start. Small, focused, and points to the official skills and rules for details.

Project Overrides – Flutter

Assume the official Flutter architecture guidance from ``docs.flutter.dev/app-architecture`` and the ``flutter-apply-architecture-best-practices`` skill in ``.agents/skills/``.

****Project-specific overrides (in effect):****

1. ****State management****: Use Bloc/Cubit (``flutter_bloc``) instead of the default ``ChangeNotifier``. When a rule or skill says "ViewModel extending ChangeNotifier", substitute "Cubit" (or "Bloc" if you have events). See ADR-0001.
2. ****Immutability****: Use ``freezed`` for domain models, API models, Bloc states, and Bloc events. Do not use manual ``dart:convert`` for models (overrides ``flutter-implement-json-serialization``). See ADR-0002.
3. ****Linting****: Use ``very_good_analysis`` in ``analysis_options.yaml``, not the default ``flutter_lints``.
4. ****Flavors****: Ship three entry points – ``main_development.dart``, ``main_staging.dart``, ``main_production.dart`` – for dev/staging/prod.
5. ****Widgets****: Views may contain no business logic. Only routing, layout, animations, and simple conditionals. Business logic goes in Bloc/Cubit.
6. ****Repositories****: Each Repository has an abstract interface and one or more concrete implementations (real, dev, staging).

Testing, folder structure, `go_router`, feature-first UI + type-first data, and everything else follow the official skill and rules.md verbatim.

6.3 The `.claude/rules/state-management.md` content (path-scoped)

Fires only when Claude reads files under `lib/ui/**/*.dart` and `test/ui/**/*.dart` [4](#):

```
---
paths:
  - "lib/ui/**/*.dart"
  - "test/ui/**/*.dart"
---

# UI Layer – State Management

- ViewModels are `Cubit<State>` or `Bloc<Event, State>`, NOT `ChangeNotifier`.
- States are `@freezed` classes with named union constructors.
- Events (for Bloc, not Cubit) are `@freezed` union classes.
- Views subscribe with `BlocBuilder`, `BlocListener`, or `BlocSelector`;
  never `ListenableBuilder` or `AnimatedBuilder` for state.
- Views dispatch events by `context.read<XBloc>().add(event)` or call
```

- a Cubit method.
- One-to-one mapping: exactly one Bloc/Cubit per View.
- Unit tests use ``bloc_test.blocTest`` to assert state sequences.

6.4 The **AGENTS.md** at project root (Devin)

Because Devin loads **AGENTS.md** automatically and Claude Code loads its own **CLAUDE.md**, the simplest way is to mirror the content: put the same content in both files. That gives you a single source of truth for the delta without either tool missing anything. [2](#)

Devin also reads `~/.claude/CLAUDE.md` as a global rule [2](#), so if a user has personal preferences there, they apply in Devin sessions too — no extra work.

7. Update & Governance Strategy

7.1 Updating the official plugin

Two mechanisms exist depending on which install path you used:

- **npx skills path** — `npx skills@1.5.17 update` refreshes both `flutter/agent-plugins` and `dart-lang/skills` [11](#).
- **Claude Code plugin path** — `claude plugin marketplace update flutter-agent-plugins` refreshes the marketplace, then reinstall the plugin to pick up the new version [3](#).

Because the delta is small and lives in the project (not the plugin), upstream refreshes never overwrite it. Only the skills, rules, and MCP config from `flutter/agent-plugins` are updated.

7.2 Managing the delta

Version the delta with the project:

- ADRs under `docs/adr/` capture the "why".
- **CHANGELOG.md** at project root tracks changes to the delta.
- Wrap the delta into a git tag when it stabilizes so downstream projects can reference it.

If and when you replicate the delta across multiple projects, promote it to Option C (a small custom plugin) and reference it from your marketplace repository.

7.3 Optional: enforcement layers

The Claude Code memory docs distinguish **soft guidance (CLAUDE.md)** from **hard enforcement (settings + hooks)**: [4](#)

"Settings rules are enforced by the client regardless of what Claude decides to do. CLAUDE.md instructions shape Claude's behavior but are not a hard enforcement layer."

If you want a hard blocker on Riverpod/GetX imports:

- Add a `PreToolUse` hook that greps writes to `lib/**` and refuses if the diff introduces those imports.
- Or use managed settings' `permissions.deny` to block specific commands or paths.

Neither of these needs a plugin — they live under `.claude/settings.json` at the project root.

8. Final Recommendation

Step 1 — Install the official Flutter plugin

From flutter/agent-plugins (Claude Code path or npx path). This gives you MVVM, layered architecture, Repository/SSOT, freezed for models, go_router, widget/integration test skills, JSON skill, layout skills, localization skill, HTTP skill — plus MCP server config and the full Dart skill catalog via dart-lang/skills.

Step 2 — Ship the small project delta

Create `./.claude/CLAUDE.md`, `./AGENTS.md`, `./.claude/rules/state-management.md`, `./.claude/rules/models.md`, `./analysis_options.yaml`, `./pubspec.yaml`, and three flavored

`main_*.dart` entry points. This encodes exactly the four things the official ecosystem does not cover (Bloc, freezed-everywhere, very_good_analysis, flavors).

Step 3 — Add auditable ADRs

Under `docs/adr/`, write ADR-0001 (state management = Bloc, overriding Flutter default), ADR-0002 (freezed for all models/states/events), ADR-0003 (three flavors). Cite the official skill and rules.md so the deviation is traceable.

Step 4 — Keep the delta in the project, not in a new plugin

Unless you plan to reuse the same Bloc override across three or more Flutter projects, the delta stays as project files. This is the smallest, cleanest, easiest-to-update shape.

Step 5 — Promote to a "bloc-delta" plugin only when reuse is real

If reuse becomes real, wrap the delta in a small Claude Code / Devin plugin. Have it declare `requiredPlugins: [flutter/agent-plugins]` in the Devin manifest so it composes automatically. Do NOT re-implement anything the official plugin already ships.

11 1

Bottom line: The official Flutter plugin ecosystem already covers ~90% of the architectural decisions. The remaining 10% (state manager, freezed everywhere, lints, flavors) is small enough that it belongs in the project itself, not in a new plugin. A custom Flutter plugin becomes worth building only when the same delta needs to be reused across multiple projects — at which point it is a *thin* wrapper on top of the official plugin, not a replacement.

Reference Reading List (all official)

- Flutter agent-plugins repo — <https://github.com/flutter/agent-plugins> ¹¹
- Flutter Agent Skills docs — <https://docs.flutter.dev/ai/agent-skills> ¹⁰
- Flutter AI Rules docs — <https://docs.flutter.dev/ai/ai-rules> ⁹
- Dart & Flutter MCP Server docs — <https://docs.flutter.dev/ai/mcp-server> ⁸

- **Official Flutter rules.md content** — <https://chromium.googlesource.com/external/github.com/flutter/flutter/+HEAD/docs/rules/rules.md> ⁷
- **flutter-apply-architecture-best-practices SKILL.md** — <https://github.com/flutter/agent-plugins/blob/main/skills/flutter-apply-architecture-best-practices/SKILL.md> ⁶
- **Claude Code Plugins Reference** — <https://code.claude.com/docs/en/plugins-reference> ⁵
- **Claude Code Memory docs** — <https://code.claude.com/docs/en/memory> ⁴
- **Claude Code Discover & install plugins** — <https://code.claude.com/docs/en/discover-plugins> ³
- **Devin CLI Rules & AGENTS.md docs** — <https://docs.devin.ai/cli/extensibility/rules> ²
- **Devin CLI Plugins docs** — <https://docs.devin.ai/cli/extensibility/plugins/overview> ¹

Everything in this report is anchored to those sources. The final architecture — install the official plugin, add a small project-level delta, and audit it via ADRs — is the smallest correct shape given exactly the composition rules Claude Code and Devin actually document.